

General Review of Hash-Based Signatures

Halil İbrahim Kaplan 

TÜBİTAK BİLGEM, Kocaeli, Turkey
Cryptanalysis Laboratory

Abstract. The advent of quantum computing threatens the security assumptions underpinning classical public-key cryptographic algorithms such as RSA and ECC. As a response, the cryptographic community has focused on developing quantum-resistant alternatives, with hash-based signature schemes emerging as a compelling option due to their reliance on well-understood hash functions rather than number-theoretic hardness assumptions. This paper presents a comprehensive review of hash-based signature schemes, covering the foundational building blocks—Lamport signatures, Winternitz one-time signatures (WOTS/WOTS+), Merkle trees, and the few-time signature scheme FORS—as well as the three standardized constructions: XMSS (RFC 8391), LMS (RFC 8554), and the stateless SPHINCS+ (FIPS 205 / SLH-DSA). A systematic comparison of these schemes highlights the trade-offs between statefulness, signature size, and implementation complexity. Furthermore, the paper analyzes several notable cryptanalytic attacks—including intermediate value guessing, Antonov’s multi-target preimage attack, fault injection, and generic multi-target attacks—examining for each the attack model, applicable constructions, complexity, and known countermeasures. By discussing both their strengths and vulnerabilities, this work provides a unified reference for understanding the design landscape of hash-based signatures as practical candidates for post-quantum digital signatures.

Keywords: Hash-Based Signatures · Post-Quantum Cryptography · Digital Signatures · Cryptographic Hash Functions

1 Introduction

Hash-based signature (HBS) schemes have been known since the late 1970s, with early contributions by Lamport and Merkle. However, for many years, they remained theoretical constructs without widespread practical use, primarily due to their large signature sizes and, in the case of stateful schemes, complex key management requirements. This perception has shifted significantly with the emergence of post-quantum cryptography (PQC), which aims to develop cryptographic primitives that are resistant to quantum attacks.

The threat posed by quantum computers—particularly Shor’s algorithm, which can break RSA and elliptic curve cryptography (ECC)—has led to renewed interest in HBS schemes. Unlike number-theoretic approaches, HBS algorithms rely solely on the security of cryptographic hash functions. While quantum algorithms such as Grover’s search reduce the effective security of hash functions (e.g., from 2^n to $2^{n/2}$ for preimage resistance), this degradation can be compensated by choosing larger parameters, and the overall security foundation of hash-based schemes remains sound against known quantum adversaries (see Table 1).

This growing interest has been formalized through standardization efforts. The Internet Engineering Task Force (IETF) published RFC 8391 for the eXtended Merkle Signature Scheme (XMSS) in 2018, followed by RFC 8554 for the Leighton-Micali Signature (LMS)

E-mail: halil.kaplan@tubitak.gov.tr (Halil İbrahim Kaplan)



scheme. In parallel, the National Institute of Standards and Technology (NIST) provided parameter recommendations and, in 2022, selected the stateless scheme SPHINCS+ as a finalist in its post-quantum cryptography standardization project. NIST subsequently published FIPS 205 (SLH-DSA) in 2024, formally standardizing SPHINCS+ as a stateless hash-based digital signature algorithm. These developments were also supported by national agencies such as ANSSI (France) and BSI (Germany), which incorporated XMSS and LMS into their respective cryptographic guidelines.

HBS schemes have since seen increasing deployment in embedded systems, firmware signing, and secure boot mechanisms—scenarios where post-quantum resilience and long-term security are critical. However, the limitations of stateful schemes like XMSS and LMS, particularly the bounded number of signatures per key, restrict their usage to environments where state can be reliably managed. Stateless alternatives such as SPHINCS+ offer more flexibility at the cost of larger signatures and slower performance.

This paper presents a comprehensive study of modern HBS schemes, focusing on the structural foundations, key and signature generation algorithms, and the design differences between XMSS and SPHINCS+. In addition to the algorithmic descriptions, we analyze the trade-offs between stateful and stateless constructions and investigate critical components such as address structures, Merkle trees, and Winternitz one-time signatures (WOTS). Furthermore, we explore known cryptanalytic approaches targeting HBS constructions, with an emphasis on multi-target attacks, fault injection strategies, and signature forgeries enabled through checksum manipulation or hash chain tampering.

The purpose of this study is to bridge theoretical constructs with practical implications, offering a clear understanding of how hash-based signatures work and how the various building blocks relate to one another in the construction of secure, practical signature schemes.

2 Basic Structures

In this section, the basic structures needed to understand hash-based signature algorithms will be explained. We begin with hash functions and signature security definitions, then describe the one-time and few-time signature building blocks (Lamport, WOTS/WOTS+, FORS), and finally explain how Merkle trees extend these to support multiple signatures.

Notation. Throughout this paper, we use w to denote the Winternitz parameter. In the WOTS/WOTS+ construction (Section 2.3), w specifies the number of bits per message chunk and each chunk can take values in $\{0, 1, \dots, 2^w - 1\}$. In the base- w representation (Section 2.6), the same parameter w now refers to the *base* of the digit representation, i.e., each digit takes values in $\{0, 1, \dots, w - 1\}$. In practice, one sets $w = 2^{w'}$ for some w' , so the two usages are consistent: a chunk of $w' = \log_2 w$ bits corresponds to exactly one base- w digit.

2.1 Hash Functions

Cryptographic hash functions are functions that convert input data of any length into a fixed-length digest value. In order for these functions to be used securely, they must satisfy certain fundamental properties. These properties are as follows:

1. **Collision Resistance:** The probability that two different inputs produce the same hash value must be negligible. In other words, finding any pair (x, y) with $x \neq y$ such that $h(x) = h(y)$ should be computationally infeasible. This property is one of the cornerstones of hash function security.
2. **Pre-image Resistance:** It must be difficult to find the input that produces a given hash value. That is, given any hash output $h(x)$, it should be computationally hard

to find the corresponding input x .

3. **Second Pre-image Resistance:** Given a specific input and its hash value, it should be difficult to find a different input that produces the same hash value. In other words, given $h(x)$, it should be computationally hard to find a different $y \neq x$ such that $h(y) = h(x)$.

Attack complexities of n -bit Hash function is given in Table 1

Table 1: Attack Complexities on an n -bit Hash

Attack	Classical Work	Quantum Work
Preimage	2^n	$2^{n/2}$
Second-Preimage	2^n	$2^{n/2}$
Collision	$2^{n/2}$	$2^{n/3}$ (Brassard–Høyer–Tapp [BHT98])

2.2 Security of Digital Signatures

Before discussing specific constructions, we briefly recall the standard security notion for digital signature schemes.

A digital signature scheme is said to achieve **Existential Unforgeability under Chosen Message Attack (EUF-CMA)** if no probabilistic polynomial-time adversary can produce a valid signature on a new message, even after obtaining signatures on adaptively chosen messages. Formally, consider the following game between a challenger and an adversary $\mathcal{A}$:

1. The challenger runs the key generation algorithm to obtain (sk, pk) and gives pk to $\mathcal{A}$.
2. $\mathcal{A}$ may adaptively query a signing oracle $\mathcal{O}_{\text{sign}}(sk, \cdot)$ on messages $M_1, \dots, M_q$ of its choice.
3. $\mathcal{A}$ outputs a pair (M^*, σ^*) .

$\mathcal{A}$ wins if σ^* is a valid signature on M^* under pk and $M^* \notin \{M_1, \dots, M_q\}$. The scheme is EUF-CMA secure if the probability that any efficient $\mathcal{A}$ wins is negligible.

All hash-based signature schemes discussed in this paper aim to achieve EUF-CMA security, which is the de facto standard for signature schemes in practice.

2.3 Lamport One-Time Signature

The Lamport signature scheme [Lam79] is the first known hash-based signature algorithm. This scheme generates signatures that can only be used once, hence requiring a different key pair for each message. The steps of the scheme are as follows:

2.3.1 Lamport Key Generation

Let us assume that a 256-bit message will be signed. In the Lamport algorithm, each bit of the signature requires 2 keys, resulting in a total of 512 public and private keys. The private keys are generated randomly and grouped as follows:

$$sk_0 = sk_{10}, sk_{20}, \dots, sk_{2560}$$

$$sk_1 = sk_{11}, sk_{21}, \dots, sk_{2561}$$

The public keys are obtained by applying a hash function to each private key:

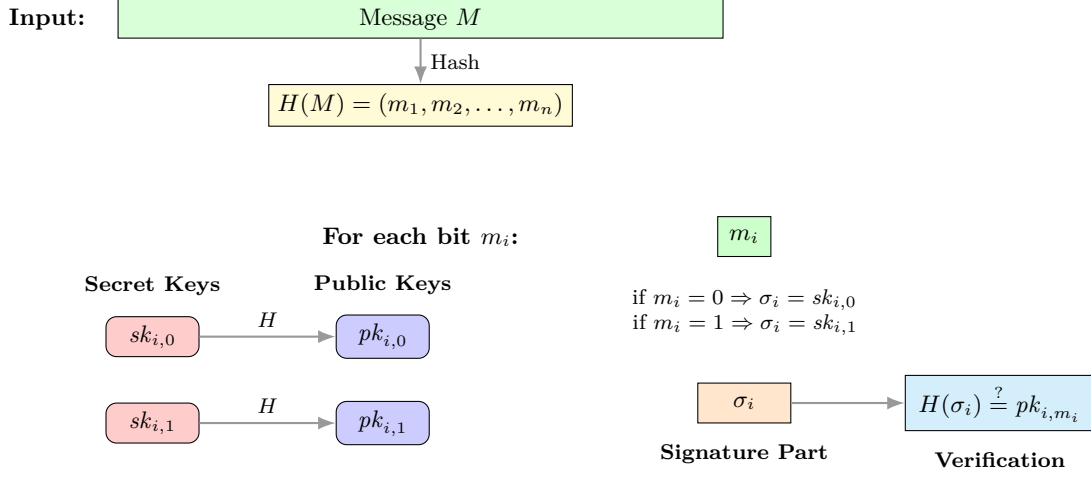


Figure 1: Lamport signature scheme for bit position i . The message M is first hashed to obtain the bit string $(m_1, \dots, m_n)$. For each bit m_i , the signer selects the corresponding secret key sk_{i,m_i} as the signature component σ_i . The verifier checks that $H(\sigma_i)$ equals the public key pk_{i,m_i} .

$$pk_0 = H(sk_{10}), H(sk_{20}), \dots, H(sk_{2560})$$

$$pk_1 = H(sk_{11}), H(sk_{21}), \dots, H(sk_{2561})$$

2.3.2 Lamport Signature

Each bit of the message is examined. If the bit is 0, the corresponding part of the signature is taken from the sk_0 group; if the bit is 1, it is taken from the sk_1 group.

For example, suppose the first three bits of our message are "001". Then, the first three parts of the signature would be:

$$sk_{10} || sk_{20} || sk_{31}$$

2.3.3 Signature Verification

The verifier examines each bit of the message one by one. If the bit value is 0, the hash of the corresponding part of the signature is computed and compared with the corresponding part of pk_0 . If the bit is 1, it is compared with the corresponding part of pk_1 .

This process is repeated for all bits. If all comparisons are equal, then the signature is successfully verified.

2.4 WOTS and WOTS+

The WOTS (Winternitz One-Time Signature) algorithm [BDE⁺13], like the Lamport signature algorithm, is a one-time signature scheme. While in the Lamport algorithm each bit of the message is signed individually, in the WOTS scheme the message is divided into chunks of w bits, and each chunk is signed instead of individual bits.

As a result, the number of signature components is reduced from n (one per bit) to $\ell_1 = \lceil n / \log_2 w \rceil$, where n is the message length in bits. However, a w -bit message chunk can take 2^w different values, and naively one would need a separate private key for each possible value.

This key-explosion problem is resolved using iterated hash chains: instead of storing 2^w distinct keys per chunk, a single secret key is hashed up to $w - 1$ times, so that the i -th value in the chain corresponds to the chunk value i . Note that the security of WOTS requires an additional checksum to prevent an adversary from advancing hash chains; the checksum mechanism is described in Section 2.8.

2.4.1 WOTS Key generation

Let us assume that the message is divided into N parts, each consisting of l bits. In this case, N secret keys $sk_0, sk_1, \dots, sk_N$ are randomly generated. The 2^l possible values that each part can take are obtained by applying a hash function repeatedly to the randomly generated keys.

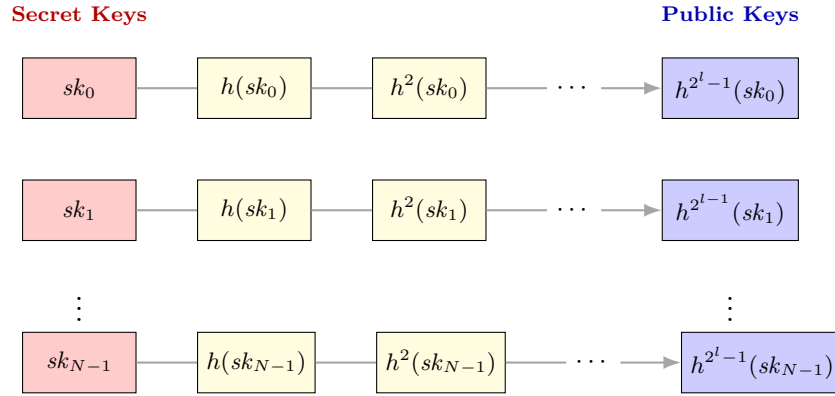


Figure 2: WOTS key generation. Each secret key sk_i (red) is iteratively hashed to produce intermediate chain values (yellow) and the final public key component $h^{2^l-1}(sk_i)$ (blue).

In Figure 2, the secret keys are shown. The public key is obtained by applying the hash function 2^l times to each secret key (i.e., the 2^l -th hash).

For example, if the value of the first message chunk is 0, then sk_0 is used in the signature. If the value is $2^l - 1$, then the signature value is $h^{2^l-1}(sk_0)$.

2.4.2 WOTS Signature

For each part of the message, the appropriate number of hash iterations is applied to the corresponding secret key. These hashed values are concatenated to form the complete signature.

For example, suppose the first three chunks of the message are:

$$001\|010\|111$$

Then the first three parts of the signature would be:

$$h(sk_0)\|h^2(sk_1)\|h^7(sk_2)$$

2.4.3 WOTS+ Signature Verification

The verifier examines each message part individually. By checking the numerical value of each chunk, the verifier determines how many hash iterations are needed to reach the public key. It then hashes the corresponding part of the signature that many times. The result is compared with the known public key. If all values match, the signature is considered valid.

2.4.4 WOTS+ [Hü17]

WOTS+ is an improved variant of WOTS that introduces a randomized *chaining function*. The chaining function c_k defines a single step in the hash chain and is given by:

$$c_k(x, r) = H(x \oplus r_k)$$

where H is the underlying hash function, x is the current chain value, and r_k is a pseudorandom bitmask derived from the public seed and the address structure for step k . The full chain of length s starting from value x_0 is computed as:

$$c^s(x_0) = c_s(c_{s-1}(\cdots c_1(x_0, r_1) \cdots, r_{s-1}), r_s)$$

This randomized masking serves two critical purposes: (1) it provides *domain separation*, ensuring that each hash evaluation within the scheme operates on a distinct, position-dependent input even when the underlying data coincides, and (2) it mitigates *multi-target attacks* by preventing an adversary from amortizing preimage searches across multiple hash chain positions.

2.5 Merkle Trees and Multi-Signatures

Lamport and WOTS schemes are one-time signature algorithms, meaning they allow signing only a single message with generated keys. However, in practical scenarios, signature algorithms are expected to support multiple signatures. To convert one-time signature schemes into algorithms that can sign multiple messages, **Merkle trees** are used.

As an example, the WOTS+ scheme can be used together with a Merkle tree. Figure 3 shows the sketch of a Merkle tree used with WOTS+. The bottom leaves of the Merkle tree are the hash values of the WOTS+ public keys. Using a Merkle tree of height h , a signature scheme capable of signing 2^h messages can be constructed.

In addition to increasing signature capacity, Merkle trees also have the advantage of **reducing public key size**. Normally, a WOTS+ public key consists of multiple components—one for each message chunk—but when used with a Merkle tree, only **one public key** is required: the root of the Merkle tree. To verify a signature, the verifier must access certain intermediate nodes of the Merkle tree in order to compute the root. These intermediate values can be included as part of the signature, thus enabling successful verification.

2.6 L-tree and Hashing public key

When combining one-time signature schemes with a Merkle tree, the public key of the one-time signature scheme is selected as the bottom leaf of the Merkle tree. However, in the WOTS algorithm, a single signature includes not one but multiple public key components—one for each message part.

To resolve this issue, two different approaches are employed:

1. **L-tree (used in XMSS)**: The L-tree is a specialized Merkle tree used to compress multiple public key components into a single public key. It repeatedly hashes pairs of public key elements until only one remains.
2. **Hashing (used in SPHINCS+)**: In this method, all public key components are concatenated, and a single hash value is computed over the entire concatenation. This resulting hash serves as the single public key for Merkle tree integration.

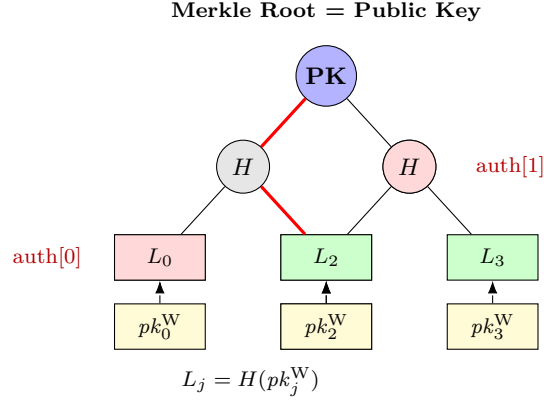


Figure 3: Merkle tree of height $h = 2$ used with WOTS+. Each leaf L_j is the hash of the corresponding WOTS+ public key pk_j^W . The root serves as the overall public key. When signing with leaf 1, the authentication path (highlighted in red) consists of sibling nodes L_0 and $H(L_2 || L_3)$, which the verifier uses to reconstruct the root.

2.7 Base- w Representation

In hash-based signature schemes such as WOTS and XMSS, the message digest is first transformed into a base- w representation before signing. This **base- w representation** is a critical component of the signing process and directly influences the structure and security of the signature.

The main purposes of using base- w representation are to:

- reduce the size of the signature compared to bit-wise representations (as in the Lamport scheme),
- represent the message digest in a form suitable for the Winternitz one-time signature construction,
- control the number of hash iterations per key chain in WOTS+,
- and balance the trade-off between performance (speed) and signature length through the choice of w .

Let the hash function produce an output of n bytes (i.e., $8n$ bits).¹ The number of base- w digits required to represent this $8n$ -bit message digest is:

$$\ell_1 = \left\lceil \frac{8n}{\log_2 w} \right\rceil$$

Each digit in this representation is an integer in the range $[0, w - 1]$, and indicates how many times the corresponding chain of the private key should be hashed to produce the signature component.

Example: Suppose $n = 256$ and $w = 16$. Then:

$$\ell_1 = \left\lceil \frac{8 \cdot 256}{\log_2 16} \right\rceil = \left\lceil \frac{2048}{4} \right\rceil = 512$$

So, the hash output will be split into 512 base-16 digits.

¹In the XMSS and SPHINCS+ specifications, n denotes the security parameter in *bytes*, not bits. Hence the factor of 8 in the numerator converts bytes to bits.

After computing the base- w representation of the message digest, a checksum is computed and appended (as described in the section 2.8), forming the complete message input for WOTS+.

The choice of w is significant:

- A smaller w increases the number of digits ℓ_1 (and thus the signature size) but reduces the number of hash computations.
- A larger w reduces the signature size but requires more hash computations per signature, increasing signing and verification time.

In summary, the base- w representation enables the WOTS+ signature scheme to efficiently encode the message digest while maintaining flexibility across a range of parameter choices.

2.8 Checksum

In hash-based signature algorithms such as WOTS and XMSS, the message to be signed is first encoded into a base- w representation. However, this encoding alone is not sufficient to ensure existential unforgeability (EUF-CMA security, see Section 2.2). Specifically, without additional protection, an adversary who observes a valid signature σ for message M could construct a valid signature for a *different* message M' by simply advancing the hash chains: if a base- w digit m_i is replaced by $m'_i > m_i$, the adversary can compute $h^{m'_i - m_i}(\sigma_i)$ to obtain the corresponding signature component. To prevent this **hash chain advancement attack**, a **checksum** value is appended to the base- w representation of the message. This checksum serves to:

- prevent attacks that attempt to alter message chunks by increasing or decreasing hash iterations,
- protect against fault injection attacks that aim to exploit the absence or corruption of the checksum,
- and detect message modifications or signature reuse.

The checksum is calculated by summing the differences between the maximum possible chunk value ($w - 1$) and each base- w digit of the message. If the message is represented by $m_0, m_1, \dots, m_{\ell_1-1}$ in base- w , the checksum c is given by:

$$c = \sum_{i=0}^{\ell_1-1} (w - 1 - m_i)$$

This value is then encoded into base- w as an additional sequence of digits, forming the checksum part of the message. The total number of checksum digits required is ℓ_2 , calculated as:

$$\ell_2 = \left\lceil \frac{\log_2(\ell_1(w - 1))}{\log_2 w} \right\rceil + 1$$

As a result, the full message to be signed by WOTS+ consists of $\ell = \ell_1 + \ell_2$ base- w digits. Each digit corresponds to the number of hash function applications to be performed on a particular chain of the private key.

The checksum plays a crucial role in maintaining the integrity and one-time nature of the WOTS+ signature. Without it, an attacker could transform a valid signature for one message into a valid signature for another message by manipulating the hash chains, which would compromise the existential unforgeability of the scheme.

2.9 FORS

The **FORS algorithm** (Forest of Random Subsets) [BHK⁺19] is a *few-time signature scheme* that serves as a critical building block in SPHINCS+ (SLH-DSA). It is used specifically at the *bottom layer* of the SPHINCS+ hypertree, where the signed data is a message digest rather than the root of another tree.

2.9.1 Why FORS Instead of WOTS+ at the Bottom Layer

In SPHINCS+, all layers above the bottom use WOTS+ instances to sign the roots of the subtrees below them. However, at the bottom layer, the scheme must sign a *message digest*. WOTS+ is a strict one-time signature: reusing a WOTS+ key pair to sign two different messages immediately leaks information that can be exploited to forge signatures. Since SPHINCS+ is stateless, it selects the signing leaf index pseudorandomly (based on a hash of the message), and there is a non-zero probability that the same leaf is selected more than once across different signing operations. FORS tolerates this because it is a *few-time* signature scheme: even if the same FORS instance signs a small number of different messages, the security degradation remains bounded and acceptable for the overall scheme's security proof.

2.9.2 FORS Construction

FORS is parameterized by two values: the number of trees k and the height a of each tree (so each tree has 2^a leaves). The message digest to be signed is split into k chunks, each of a bits, yielding k integer indices $i_0, i_1, \dots, i_{k-1}$, where each $i_j \in \{0, 1, \dots, 2^a - 1\}$.

Key Generation. For each of the k trees, 2^a secret key values are generated (derived deterministically from a seed and an address structure). The leaves of each tree are computed by hashing the corresponding secret keys. A binary Merkle tree is then built over these 2^a leaves, yielding a root for each tree. The k tree roots are combined (e.g., by hashing their concatenation) to produce the overall FORS public key.

Signing. Given the k indices $(i_0, \dots, i_{k-1})$ derived from the message digest, the FORS signature consists of, for each tree j :

- the secret key value sk_{j,i_j} at position i_j in tree j , and
- the authentication path (the a sibling nodes along the path from leaf i_j to the root of tree j).

Verification. For each tree j , the verifier hashes the provided secret key to obtain the leaf, then uses the authentication path to reconstruct the tree root. All k reconstructed roots are combined and compared against the known FORS public key.

2.9.3 Example

Consider FORS with $k = 4$ trees of height $a = 3$ (so each tree has $2^3 = 8$ leaves). Suppose the message digest yields the following four 3-bit chunks:

100 011 101 000

This means: in tree 0, select leaf 4 (100_2); in tree 1, select leaf 3 (011_2); in tree 2, select leaf 5 (101_2); in tree 3, select leaf 0 (000_2). Figure 4 illustrates the resulting signature structure.

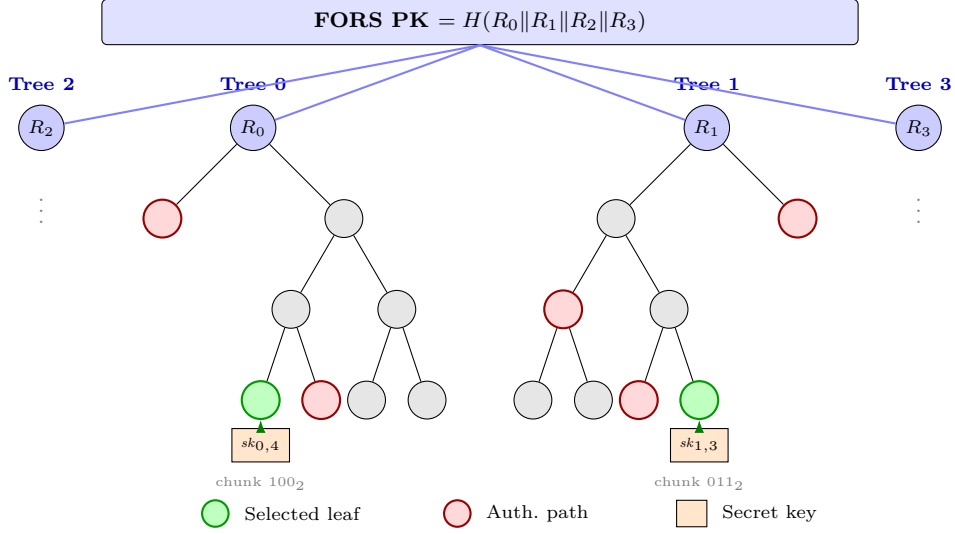


Figure 4: FORS signature example with $k = 4$ trees, each of height $a = 3$. Trees 0 and 1 are shown in detail: the message digest chunk selects a leaf index (green), and the authentication path siblings (red) are included in the signature along with the secret key $sk_{i,idx}$. All four tree roots $R_0, \dots, R_3$ are hashed together to form the FORS public key.

2.10 Comparison of OTS and FTS Building Blocks

Table 2 compares the one-time and few-time signature schemes discussed above. Understanding the trade-offs between these building blocks explains why specific components are chosen for different standardized schemes: WOTS+ is used in XMSS, LMS, and the upper layers of SPHINCS+ due to its compact signatures, while FORS is used at the bottom layer of SPHINCS+ because it tolerates limited key reuse.

Table 2: Comparison of one-time and few-time signature building blocks

Property	Lamport	WOTS	WOTS+	FORS
Type	OTS	OTS	OTS	FTS
Sig. components	$2n$	ℓ	ℓ	$k(a + 1)$
Sig. size	$2n \cdot n$	$\ell \cdot n$	$\ell \cdot n$	$k(a + 1) \cdot n$
Hash calls (sign)	0	$\sum m_i$	$\sum m_i$	k (+ auth)
Checksum	No	Yes	Yes	No
Randomization	None	None	XOR bitmask	Tweakable
Multi-target	Vulnerable	Vulnerable	Resistant	Resistant
Key reuse	Insecure	Insecure	Insecure	Bounded
Used in	(Historical)	–	XMSS, LMS, SPHINCS+	SPHINCS+

Here, n is the security parameter (hash output length), $\ell = \ell_1 + \ell_2$ is the total number of WOTS/WOTS+ chains, k is the number of FORS trees, and a is the height of each FORS tree.

2.11 Address Structures in Hash-Based Signature Schemes

Hash-based signature schemes consist of either very large Merkle trees or multi-layered Merkle trees in order to increase their signature capacity. Since the same hash function is repeatedly used within the tree and similar operations are performed, **address structures** are used in hash-based signature schemes to:

- distinguish each operation,
- determine the position within the tree,
- track how many hash operations have been applied,
- and provide input differentiation for the hash functions.

Table 3 shows the **OTS address structure** used in the XMSS algorithm [HBG⁺18]. In the example structure, the following fields appear in order from top to bottom:

- the layer in the multi-tree,
- the specific tree identifier,
- address type (e.g., OTS),
- index of the L-tree–WOTS pair,
- WOTS chain identifier,
- index of the hash operation within the chain,
- a flag indicating whether the hash value is used for encryption or masking.

Depending on the algorithm and its internal structure, the format of the address field can vary.

Table 3: XMSS OTS Address Format

Field	Size
layer address	32 bits
tree address	64 bits
type = 0	32 bits
OTS address	32 bits
chain address	32 bits
hash address	32 bits
keyAndMask	32 bits

2.12 Stateful and Stateless Hash-Based Signature Schemes

Hash-based signature algorithms can be categorized into two types: **stateful** and **stateless**. The fundamental difference between these two categories lies in the management of state information during the signature generation and verification processes.

Stateful hash-based signature algorithms require the maintenance of state information while generating signatures. These algorithms rely on *one-time keys* that can be used only once per signature operation. Reusing these keys leads to serious security vulnerabilities. A representative example of such an algorithm is XMSS [HBG⁺18].

Stateless hash-based signature algorithms, in contrast, do not require any state information to be stored during the signing process. A prominent example of a stateless algorithm is SPHINCS+ [BHK⁺19].

3 XMSS and $XMSS^{MT}$

In this section, the XMSS/ $XMSS^{MT}$ algorithms, as defined in the documents [CAD⁺20] [HBG⁺18], will be explained using the fundamental structures presented in Section 2. The architecture of XMSS is given in Figure 5.

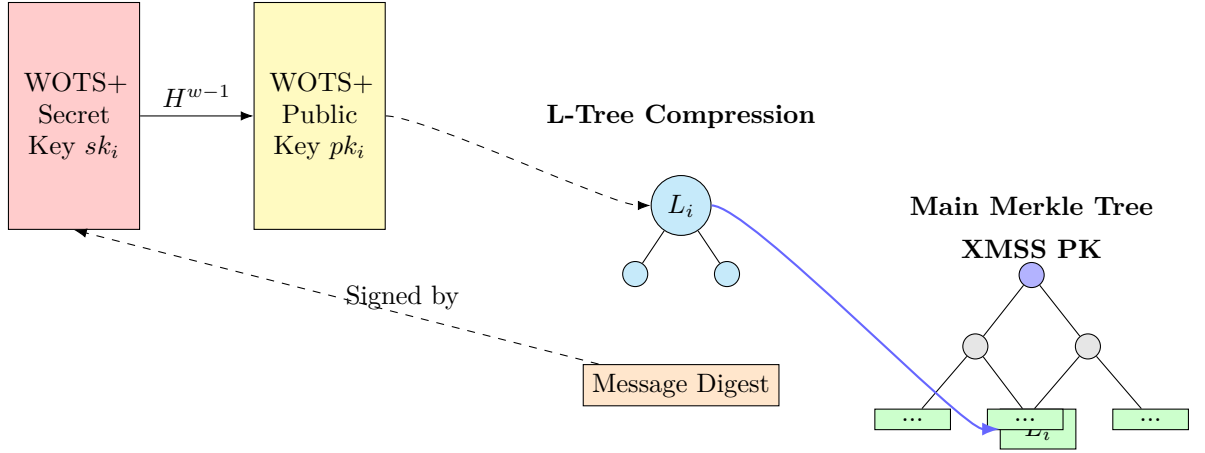


Figure 5: The architecture of XMSS. A WOTS+ secret key sk_i (consisting of ℓ random n -byte strings) is used to sign a message digest. The corresponding WOTS+ public key pk_i (obtained by hashing each chain $w-1$ times) is compressed via an L-Tree into a single leaf L_i . This leaf is placed in a Merkle tree of height h , whose root serves as the overall XMSS public key.

3.1 XMSS Key Generation

In Algorithm 1, a pseudo-code representation of the key generation process of the **XMSS** algorithm is provided. First, the **WOTS+** private keys are generated. These keys can be created either completely at random or by passing a seed value through a key generation function. If all keys are generated randomly, they must be stored in memory for the signing process. However, since storing all the keys in memory is not realistic, this method is generally not used. Next, certain seed values are generated to be used throughout the algorithm, and the hash values of the WOTS+ private keys are taken to produce the WOTS+ public keys. In the pseudo-code, the value **idx** at the beginning of the **SK** section represents the state information (the index of the next unused OTS key pair).

The key functions used in the algorithm are:

- **WOTS_genSK()**: Generates a WOTS+ secret key, consisting of ℓ random n -byte strings (one per hash chain).
- **setSK_PRF(SK, SK_PRF)**: Stores the pseudorandom function key **SK_PRF** (an n -byte string) into the private key structure. This key is used during signing to generate randomized message hashes.

- **setSEED**(SK, SEED): Stores the public seed **SEED** (an n -byte string) in the key structure. The seed is used as a public parameter for all hash computations (bitmasks, address-based hashing).
- **treeHash**(SK, s , t , ADRS): Computes the root of the Merkle subtree starting at leaf index s with height t , using the WOTS+ public keys as leaves (after L-tree compression). The address structure **ADRS** tracks the position within the tree hierarchy.
- **toByte**(x , y): Converts the integer x into a y -byte string.

Algorithm 1 XMSS_keyGen – Generate an XMSS key pair

Input: None

Output: XMSS private key SK, XMSS public key PK

```

1:  // Example initialization for SK-specific contents
2:  idx ← 0
3:  for  $i = 0$  to  $2^h - 1$  do
4:    wots_sk[i] ← WOTS_genSK()
5:  end for
6:  Initialize SK_PRF with a uniformly random  $n$ -byte string
7:  setSK_PRF(SK, SK_PRF)
8:  // Initialization for common contents
9:  Initialize SEED with a uniformly random  $n$ -byte string
10: setSEED(SK, SEED)
11: setWOTS_SK(SK, wots_sk)
12: ADRS ← toByte(0, 32)
13: root ← treeHash(SK, 0, h, ADRS)
14: SK ← idx || wots_sk || SK_PRF || root || SEED
15: PK ← OID || root || SEED
16: return SK || PK

```

3.2 XMSS Sign

In Algorithm 2, the pseudo-code of the XMSS signing algorithm is presented. The **treeSig** function signs the message using WOTS and generates the authentication path required to reach the root of the tree. First, the state information is retrieved, then a random value is generated and concatenated with the message, after which a hash is computed from this combination. This hash value is then signed using the **treeSig** algorithm. The final signature consists of the concatenation of the state information, the random value, and the **treeSig** signature, respectively.

3.3 XMSS Verify

In Algorithm 3 and 4, the pseudo-code of the XMSS signature verification is presented. First, the WOTS public key is derived from the message. Then, these WOTS public keys are passed through the **L-tree** to obtain the leaves of the Merkle tree. From the leaves of the Merkle tree, the root is reached to obtain the XMSS public key and perform verification. If the derived public key matches the expected value, the signature is considered valid. The authentication path contained in the signature is used to climb to the root in both the **L-tree** and the Merkle tree.

Algorithm 2 XMSS_sign – Generate an XMSS signature and update the XMSS private key

Input: Message M , XMSS private key SK

Output: Updated SK , XMSS signature Sig

```

1:  idx_sig ← getIdX(SK)
2:  setIdX(SK, idx_sig + 1)
3:  ADRS ← toByte(0, 32)
4:  r ← PRF(getSK_PRF(SK), toByte(idx_sig, 32))
5:  M' ← H_msg(r || getRoot(SK) || toByte(idx_sig, n), M)
6:  Sig ← idx_sig || r || treeSig(M', SK, idx_sig, ADRS)
7:  return SK || Sig

```

Algorithm 3 XMSS_rootFromSig: Compute a root node from a tree signature

Input:

index idx_sig

WOTS+ signature sig_ots

authentication path $auth$

n -byte message M'

seed $SEED$

address $ADRS$

Output: n -byte root value $node[0]$

```

1:  ADRS.setType(0) // Type = OTS hash address
2:  ADRS.setOTSAddress(idx_sig)
3:  pk_ots ← WOTS_pkFromSig(sig_ots, M', SEED, ADRS)
4:  ADRS.setType(1) // Type = L-tree address
5:  ADRS.setLTreeAddress(idx_sig)
6:  Initialize node[0] and node[1] as  $n$ -byte arrays
7:  node[0] ← ltree(pk_ots, SEED, ADRS)
8:  ADRS.setType(2) // Type = hash tree address
9:  ADRS.setTreeIndex(idx_sig)
10: for  $k = 0$  to  $h - 1$  do
11:   ADRS.setTreeHeight(k)
12:   if  $\lfloor idx\_sig / 2^k \rfloor \bmod 2 = 0$  then
13:    ADRS.setTreeIndex(ADRS.getTreeIndex() / 2)
14:    node[1] ← RAND_HASH(node[0], auth[k], SEED, ADRS)
15:   else
16:    ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2)
17:    node[1] ← RAND_HASH(auth[k], node[0], SEED, ADRS)
18:   end if
19:   node[0] ← node[1] // Update current node
20: end for
21: return node[0]

```

Algorithm 4 XMSS_verify – Verify an XMSS signature using the corresponding XMSS public key and a message

Input: XMSS signature Sig , message M , XMSS public key PK

Output: Boolean

```

1:  ADRS  $\leftarrow$  toByte(0, 32)
2:   $M' \leftarrow H_{\text{msg}}(r \parallel \text{getRoot}(\text{PK}) \parallel \text{toByte}(\text{idx\_sig}, n), M)$ 
3:  node  $\leftarrow \text{XMSS\_rootFromSig}(\text{idx\_sig}, \text{sig\_ots}, \text{auth}, M', \text{getSEED}(\text{PK}), \text{ADRS})$ 
4:  if node == getRoot(PK) then
5:      return true
6:  else
7:      return false
8:  end if

```

3.4 XMSS^{MT}

XMSS^{MT} is the multi-tree version of the XMSS algorithm. While the XMSS algorithm consists of a single large tree, the multi-tree structure comprises many smaller trees organized in d layers, each of height $h' = h/d$, where h is the total tree height. Each tree at layer j uses its WOTS+ instances to sign the roots of the trees at layer $j - 1$, and the bottom-layer trees (layer 0) sign the actual messages. The advantage of this approach is that signing is significantly faster: instead of computing a single Merkle tree of height h (requiring 2^h leaf computations), only d smaller trees of height h' need to be computed, reducing the per-signature cost [HBG⁺18]. In a multi-tree structure, the signature is formed by the combination of the signatures and authentication paths of each layer. Figure 6a shows the multi-tree signature structure. Figure 6b illustrates the overall structure of the multi-tree.

3.5 Tree Traversal

A practical challenge in implementing XMSS and XMSS^{MT} is the efficient computation of authentication paths. A naïve approach would require storing all 2^h leaves or recomputing them for each signature, which is prohibitive for large trees.

The **BDS (Buchmann–Dahmen–Schneider) traversal algorithm** [BDE⁺13] addresses this by maintaining a small set of precomputed nodes and updating them incrementally as signatures are generated. The key idea is to use a combination of:

- **Treeshash instances:** background computations that prepare upcoming authentication path nodes before they are needed.
- **Retain stack:** a small stack storing nodes that would be expensive to recompute later.
- **Keep array:** nodes from the current authentication path that will be needed in future paths.

The BDS algorithm achieves a time complexity of $O(h/2)$ hash computations per signature update and requires $O(h)$ storage for the traversal state. The XMSS RFC [HBG⁺18] specifies a variant of this algorithm as the recommended traversal strategy.

For XMSS^{MT} , traversal is performed independently within each subtree of height h' , and a new subtree is initialized only when the current one is exhausted, further reducing the per-signature cost.

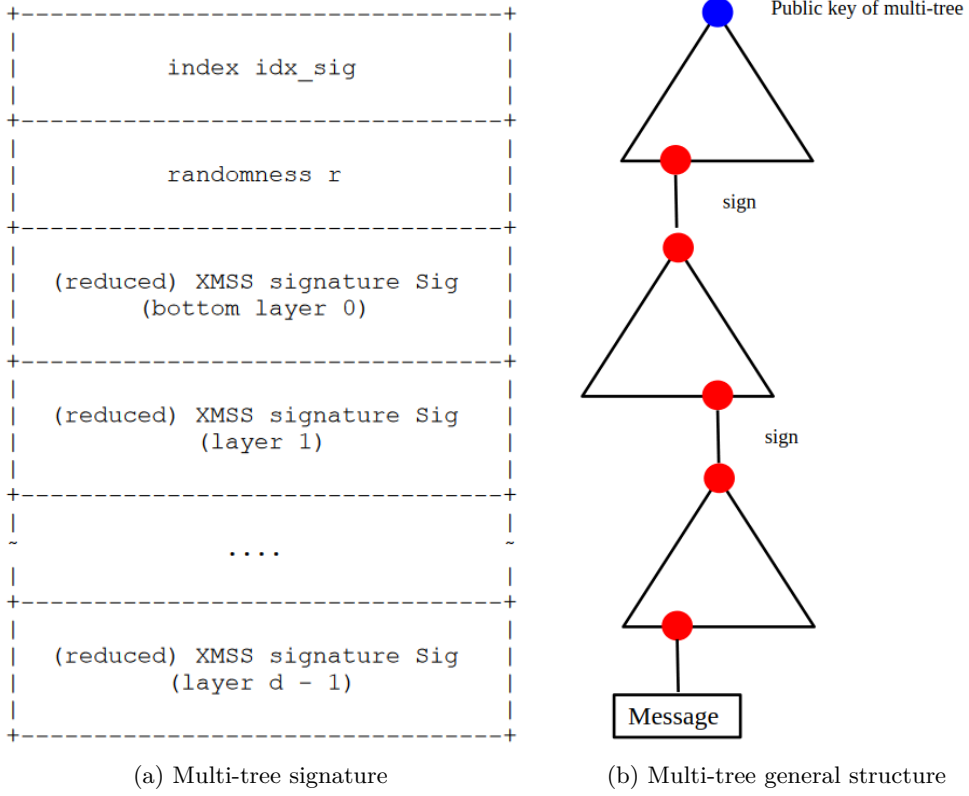


Figure 6: XMSS^{MT} multi-tree structure with d layers, each containing subtrees of height h' . (a) The multi-tree signature consists of d XMSS signatures, one per layer. (b) The overall hypertree: each subtree at layer j signs the root of a subtree at layer $j-1$. For example, with $h = 20$ and $d = 4$, each subtree has height $h' = 5$, supporting 2^{20} total signatures.

4 SPHINCS+

SPHINCS+ [NIS24] is one of the three signature algorithms standardized by NIST (published as FIPS 205 / SLH-DSA in 2024). Structurally, it is very similar to XMSS. Fundamentally, there are two main differences. First, the SPHINCS+ algorithm does not use an L-tree structure. Instead, it transforms WOTS+ public keys into a single public key by applying a hash function (see Section 2.6). Second, unlike XMSS, which contains only a WOTS+ signature, the SPHINCS+ algorithm also includes a FORS signature layer underneath the WOTS+ signature. Since the FORS signature algorithm is a *few-time* signature, it tolerates the possibility of signing multiple messages with the same key pair (see Section 2.8 for details). For this reason, SPHINCS+ is a stateless algorithm. Before each signing operation, it pseudorandomly selects an index (derived from a hash of the message) and generates the signature and authentication path from the part of the tree corresponding to that index. Because it does not require securely storing and updating state information, stateless SPHINCS+ offers a significant advantage over XMSS in this regard. However, the FORS layer, which makes SPHINCS+ stateless, adds an additional layer of signature under WOTS+, thereby increasing the signature size compared to XMSS. Note that while both XMSS and SPHINCS+ use tree-based structures, their public key sizes are not necessarily identical and depend on the specific parameter sets chosen (e.g., security level, hash function). Figure 9 illustrates the general structure of the SPHINCS+ algorithm.

4.1 SPHINCS+ Parameters

Parameters and their descriptions are given below:

n : Hash output size in bits. All outputs of the underlying hash functions (e.g., H_{msg} , PRF, and Merkle-node hashes) are exactly n bits long.

w : Winternitz parameter. Determines the base for WOTS+ chains. Higher w reduces signature length at the cost of more hash computations per chain.

ℓ : Total number of n -bit chains in each WOTS+ key or signature, given by

$$\ell = \ell_1 + \ell_2, \quad \ell_1 = \left\lceil \frac{8n}{\log_2 w} \right\rceil, \quad \ell_2 = \left\lfloor \frac{\log_2(\ell_1(w-1))}{\log_2 w} \right\rfloor + 1.$$

h : Total height of the Merkle hypertree. The overall number of leaves (XMSS key pairs) is 2^h .

h' : Height of each XMSS subtree. Each subtree contains $2^{h'}$ leaves, and we arrange d such layers to reach height h .

d : Number of layers in the hypertree, satisfying

$$d = \frac{h}{h'}.$$

k : Number of parallel FORS trees. The n -bit message digest is split into k indices, each signed by one FORS tree.

SK.seed	n bytes	PK.seed	n bytes
SK.prf	n bytes	PK.root	n bytes
PK.seed	n bytes		
PK.root	n bytes		

Figure 8: SPHINCS+ public key

4.2 SPHINCS+ Key Generation

Key generation derives a full SPHINCS+ key pair from three seeds: the secret seed, the PRF key, and the public seed. It first initializes the ADRS value with a zero-filled 32-byte string and configures it for the top-level XMSS tree by setting the layer address to $d - 1$. The root node of that tree is computed via a single call to `xmss_node`, which absorbs the secret seed, the public seed, and the ADRS state. Finally, the algorithm returns the secret key tuple $(SK.seed, SK.prf, PK.seed, PK.root)$ alongside the public key $(PK.seed, PK.root)$ (see Algorithm 5).

Algorithm 5 SPHINCS_keyGen – Generate a SPHINCS+ key pair

Input: None

Output: SPHINCS+ private key **SK**, SPHINCS+ public key **PK**

- 1: Generate uniformly random n -byte strings: **SK.SEED**, **SK.PRF**, **PK.SEED**
 - 2: Initialize ADRS as a zeroed address structure
 - 3: **PK.root** $\leftarrow$ `treehash(SK_SEED, PUB_SEED, ADRS)`
 - 4: **SK** $\leftarrow$ **SK.SEED** $\parallel$ **SK.PRF** $\parallel$ **PUB.SEED** $\parallel$ **PK.root**
 - 5: **PK** $\leftarrow$ **PK.SEED** $\parallel$ **PK.root**
 - 6: **return** **SK**, **PK**
-

4.3 SPHINCS+ Sign

Signature generation begins by initializing the ADRS to zero and optionally randomizing the message randomizer input. The PRF keyed by *SK.prf* produces the randomness R that is prepended to the signature. A message digest is then computed over R , the public seed, the root, and the message itself. This digest is split into the message digest bits md and the tree/leaf indices idx_tree, idx_leaf . The FORS scheme is applied at the lowest layer (layer 0) to sign md , yielding SIG_FORS and an authenticated public key PK_FORS . Finally, the hypertree (HT) signature over PK_FORS is computed and appended, producing the full SPHINCS+ signature (see Algorithm 6)

Table 4: SPHINCS+ signature data format

Signature
Randomness R
FORS Signature SIG_{FORS}
HT Signature SIG_{HT}

4.4 SPHINCS+ Verify

Verification mirrors signing: it parses the incoming signature into R , SIG_FORS , and SIG_HT . The verifier recomputes the message digest and splits it into md , idx_tree , and idx_leaf . It then re-derives the FORS public key by running *fors_pkFromSig* on

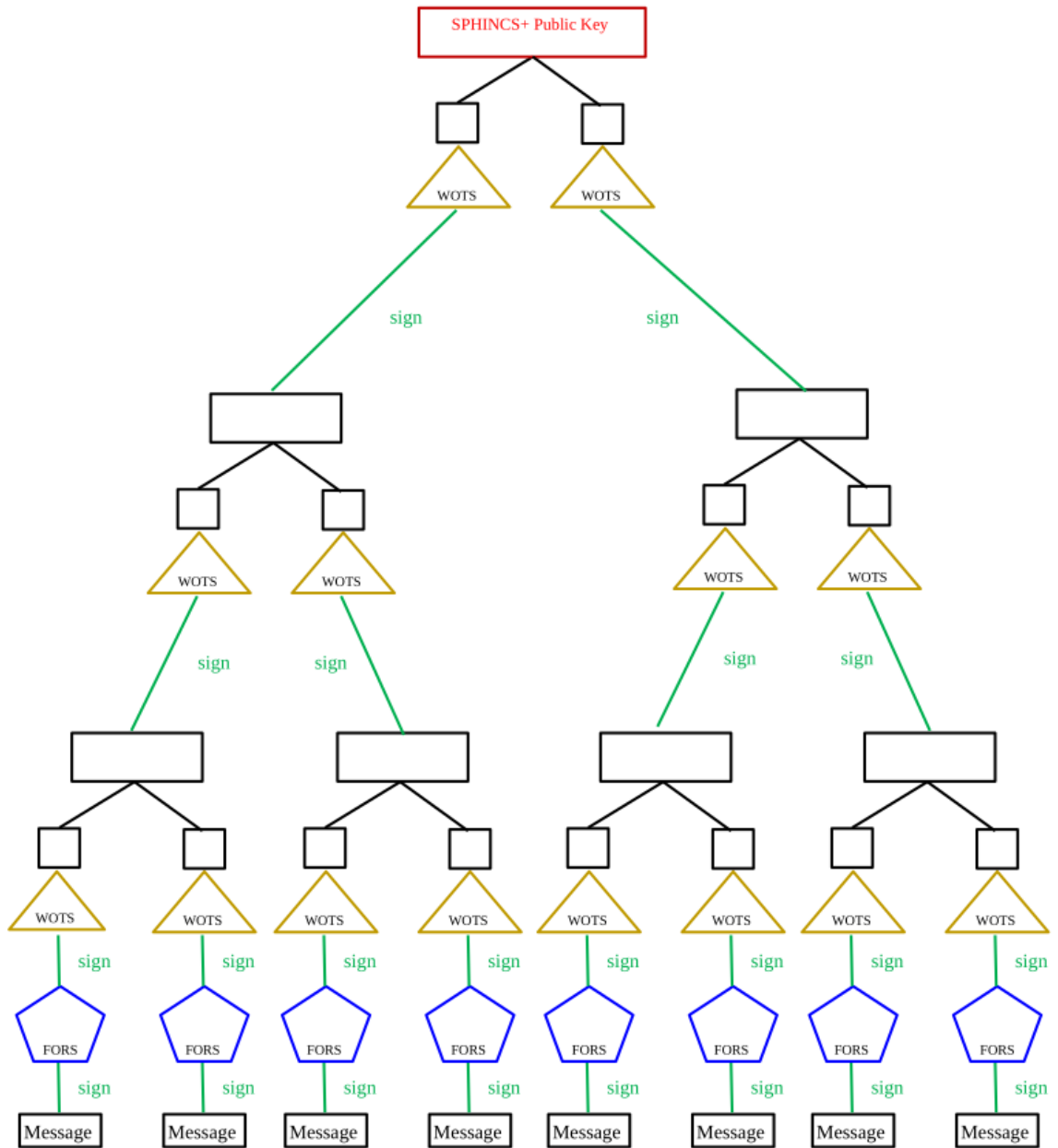


Figure 9: SPHINCS+ General structure

Algorithm 6 `spx_sign` – Generating a SPHINCS+ signature

Input: Message M , private key $SK = (SK.seed, SK.prf, PK.seed, PK.root)$
Output: SPHINCS+ signature SIG

```

1:  ADRS  $\leftarrow$  toByte(0, 32)
2:  opt  $\leftarrow$  PK.seed
3:  if RANDOMIZE then
4:    opt  $\leftarrow$  rand(n)
5:  end if
6:  R  $\leftarrow$  PRF_msg(SK.prf, opt, M)
7:  SIG  $\leftarrow$  R
8:  digest  $\leftarrow$  H_msg(R, PK.seed, PK.root, M)
9:  tmp_md  $\leftarrow$  first  $\lfloor (ka + 7)/8 \rfloor$  bytes of digest
10: tmp_idx_tree  $\leftarrow$  next  $\lfloor (h - h/d + 7)/8 \rfloor$  bytes of digest
11: tmp_idx_leaf  $\leftarrow$  next  $\lfloor (h/d + 7)/8 \rfloor$  bytes of digest
12: md  $\leftarrow$  first  $ka$  bits of tmp_md
13: idx_tree  $\leftarrow$  first  $(h - h/d)$  bits of tmp_idx_tree
14: idx_leaf  $\leftarrow$  first  $h/d$  bits of tmp_idx_leaf
15: ADRS.setLayerAddress(0)
16: ADRS.setTreeAddress(idx_tree)
17: ADRS.setType(FORS_TREE)
18: ADRS.setKeyPairAddress(idx_leaf)
19: SIG_FORS  $\leftarrow$  fors_sign(md, SK.seed, PK.seed, ADRS)
20: SIG  $\leftarrow$  SIG || SIG_FORS
21: PK_FORS  $\leftarrow$  fors_pkFromSig(SIG_FORS, md, PK.seed, ADRS)
22: ADRS.setType(TREE)
23: SIG_HT  $\leftarrow$  ht_sign(PK_FORS, SK.seed, PK.seed, idx_tree, idx_leaf)
24: SIG  $\leftarrow$  SIG || SIG_HT
25: return SIG

```

SIG_FORS and md . Finally, the hypertree signature is checked with ht_verify , which returns *true* only if every component matches the stored public root (see Algorithm 7).

Algorithm 7 `spx_verify` – Verify a SPHINCS+ signature SIG on a message M using a SPHINCS+ public key PK

Input: Message M , signature SIG , public key PK

Output: Boolean

```

1:   $ADRS \leftarrow \text{toByte}(0, 32)$ 
2:   $R \leftarrow SIG.\text{getR}()$ 
3:   $SIG\_FORS \leftarrow SIG.\text{getSIG\_FORS}()$ 
4:   $SIG\_HT \leftarrow SIG.\text{getSIG\_HT}()$ 
5:   $\text{digest} \leftarrow H\_msg(R, PK.\text{seed}, PK.\text{root}, M)$ 
6:   $\text{tmp\_md} \leftarrow \text{first } \lfloor (ka + 7)/8 \rfloor \text{ bytes of } \text{digest}$ 
7:   $\text{tmp\_idx\_tree} \leftarrow \text{next } \lfloor (h - h/d + 7)/8 \rfloor \text{ bytes of } \text{digest}$ 
8:   $\text{tmp\_idx\_leaf} \leftarrow \text{next } \lfloor (h/d + 7)/8 \rfloor \text{ bytes of } \text{digest}$ 
9:   $md \leftarrow \text{first } ka \text{ bits of } \text{tmp\_md}$ 
10:  $\text{idx\_tree} \leftarrow \text{first } (h - h/d) \text{ bits of } \text{tmp\_idx\_tree}$ 
11:  $\text{idx\_leaf} \leftarrow \text{first } h/d \text{ bits of } \text{tmp\_idx\_leaf}$ 
12:  $ADRS.\text{setLayerAddress}(0)$ 
13:  $ADRS.\text{setTreeAddress}(\text{idx\_tree})$ 
14:  $ADRS.\text{setType}(FORS\_TREE)$ 
15:  $ADRS.\text{setKeyPairAddress}(\text{idx\_leaf})$ 
16:  $PK\_FORS \leftarrow \text{fors\_pkFromSig}(SIG\_FORS, md, PK.\text{seed}, ADRS)$ 
17:  $ADRS.\text{setType}(TREE)$ 
18: return  $ht\_verify(PK\_FORS, SIG\_HT, PK.\text{seed}, \text{idx\_tree}, \text{idx\_leaf}, PK.\text{root})$ 
```

5 LMS and HSS

The **Leighton–Micali Signature (LMS)** scheme [MCF19] is a stateful hash-based signature algorithm standardized in IETF RFC 8554. Like XMSS, LMS is built from a one-time signature component combined with a Merkle tree to support multiple signatures. Its multi-tree extension is called **HSS (Hierarchical Signature System)**, analogous to XMSS^{MT}.

5.1 LMS Construction

LMS uses the **Leighton–Micali One-Time Signature (LM-OTS)** as its underlying OTS scheme. LM-OTS operates similarly to WOTS: the message digest is divided into p chunks in base 2^w , and each chunk determines how many times a hash chain is iterated. The key differences from WOTS+ are:

- **Randomized hashing:** LM-OTS uses a different randomization approach. Instead of XOR-based bitmasking (as in WOTS+), LM-OTS prepends a diversification string (the OTS type identifier, the key identifier, the leaf index q , and a chain index i) to each hash input, achieving domain separation through message formatting rather than XOR masking.
- **Final hash:** After computing all p chain values, LM-OTS concatenates them and applies a final hash to produce a single n -byte value that serves as the OTS public key. WOTS+ uses an L-tree for this compression step in XMSS.

The LMS Merkle tree is constructed over 2^h LM-OTS public keys (where h is the tree height), and the LMS public key is the root of this tree. Signing uses the q -th OTS key pair and includes the authentication path. Verification reconstructs the root from the OTS signature and checks it against the stored public key.

5.2 HSS: Multi-Tree Extension

HSS extends LMS to a multi-tree (hypertree) structure with L levels, analogous to XMSS^{MT}. At each level i ($i = 0, \dots, L-1$), there is an LMS tree of height h_i . The top-level tree signs the public keys of the next level down, and the bottom level signs the actual messages. The total number of available signatures is $\prod_{i=0}^{L-1} 2^{h_i}$.

5.3 Comparison of Standardized HBS Schemes

Table 5 provides a systematic comparison of the three standardized hash-based signature schemes.

Table 5: Comparison of standardized hash-based signature schemes

Property	LMS/HSS	XMSS/XMSS ^{MT}	SPHINCS+
Standard	RFC 8554	RFC 8391	FIPS 205 (SLH-DSA)
Statefulness	Stateful	Stateful	Stateless
OTS Scheme	LM-OTS	WOTS+	WOTS+ + FORS
Randomization	Domain separation	XOR bitmask	Tweakable hash
PK Compression	Final hash	L-tree	Hash of concat.
Multi-tree	HSS (L levels)	XMSS ^{MT} (d layers)	Hypertree (d layers)
Max Signatures	$\prod 2^{h_i}$	2^h	Unlimited
PK Size	$2n + 24$ bytes	$2n + 4$ bytes (OID)	$2n$ bytes
Sig. Size (128-bit)	$\approx 2-4$ KB	$\approx 2-3$ KB	$\approx 7-50$ KB
Security Assumption	PRF, 2nd-preimage	PRF, 2nd-preimage	PRF, multi-target

Key observations:

- LMS/HSS and XMSS/XMSS^{MT} are both stateful and produce compact signatures, but require careful state management to avoid catastrophic key reuse.
- SPHINCS+ eliminates state management at the cost of significantly larger signatures (due to the FORS layer and deeper hypertree).
- The choice of domain separation mechanism (XOR bitmask in XMSS vs. input formatting in LMS) leads to slightly different security proofs but comparable security guarantees.
- For applications where state can be securely managed (e.g., firmware signing, secure boot), LMS or XMSS are preferred due to their smaller signatures. For general-purpose applications, SPHINCS+ is the safer choice.

6 Some Attacks on Hash-Based Signatures

This section examines several notable attacks on hash-based signature algorithms. For each attack, we present the attack model, the construction, the applicable schemes, the complexity, and known countermeasures.

6.1 Intermediate Value Guessing Attack

This attack was defined in the paper by Booth et al. [BXKSN21]. In the document by Hülsing et al. [HBG⁺18], it is stated that the secret key values used in the WOTS+ signature must be generated in a uniformly distributed manner, but the specific generation method is left to the implementer.

We first introduce the notation used in this section. Let osk (*OTS secret key*) denote the WOTS+ secret key for leaf index i , consisting of ℓ chain starting values. Let F denote a pseudorandom function used to derive these values, and let w denote the Winternitz parameter (so each base- w digit ranges over $\{0, 1, \dots, w-1\}$). A digit value of 0 means the secret key element is used directly (without any hash iteration), while a digit value of $w-1$ (denoted F in the base- w representation) means the element is hashed $w-1$ times, yielding the public key component.

Assume that the WOTS+ secret keys (osk) are generated from the seed values $seed_{OTS,i}$ as follows:

$$osk = (x_{i,1}, x_{i,2}, \dots, x_{i,\ell}) = (F(seed_{OTS,i}, 1), \dots, F(seed_{OTS,i}, \ell))$$

Under this assumption:

- The signatures of messages whose base- w representation includes zero values leak the corresponding indices of osk . For instance, if the base- w representation of message M is $BM = (b_1, 0, b_2)$, then the signature $\sigma_M = (Z_1, x_{i,2}, Z_3)$ reveals $x_{i,2}$. (Here, each Z value is derived by applying the hash algorithm to a secret key index w times.)
- The osk values used to sign different messages are generated from different seed values.
- Using a different seed for each WOTS+ secret key allows a multi-target attack, as described below.

In this attack, the adversary proceeds as follows:

- Executes the signing oracle to obtain multiple WOTS+ message-signature pairs.
- Filters out pairs where fewer than two osk values are revealed.
- Assumes that a signature has the form $\sigma = (Z_1, \dots, Z_\ell)$ and the message base- w representation is $BM = (b_1, \dots, b_\ell)$. The adversary identifies two indices k_1 and k_2 such that Z_1 and Z_2 correspond to x_{i,k_1} and x_{i,k_2} respectively. (Here, k_1 and k_2 are the positions where the base- w value is zero.)
- From this, the adversary learns:

$$x_{i,k_1} = F(seed_{OTS,i}, k_1), \quad x_{i,k_2} = F(seed_{OTS,i}, k_2)$$

- Using these values, the adversary constructs tables from message-signature pairs (assuming layer j is known):

$$(x_{k_1,k_2}, x_{k_2,i}, j), \quad (x_{k_3,k_4}, x_{k_4,i}, j)$$

- For each guessed $seed_{OTS,i}$, the adversary reconstructs a candidate osk' and checks whether the generated value matches existing entries in the table. If both x_{k_1} and x_{k_2} match, the guess for $seed_{OTS,i}$ is considered correct.

Assuming the adversary correctly guesses $seed_{OTS,i}$, let the message M_i have the following XMSSMT signature structure. To forge a valid signature, the adversary uses one of the following strategies:

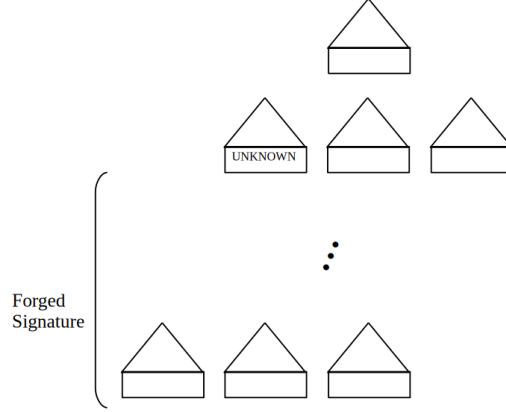


Figure 10: Intermediate Value Guessing: the adversary recovers OTS seed values from observed signatures and uses them to forge signatures at the corresponding tree layer.

Case 1: Seed Belongs to Layer 0 At layer 0, the signature is generated by signing $H(R \parallel \text{ROOT} \parallel i, M_i)$. Let M_j be the adversary’s chosen message. The adversary computes $H(R \parallel \text{ROOT} \parallel i, M_j)$, signs it with the recovered secret key, and replaces the original signature value with this forged one. The authentication path values (**Auth**) remain unchanged, as they depend on the secret key used, not the signed message.

Case 2: Seed Belongs to Layer $j > 0$ Suppose the recovered seed belongs to a WOTS+ instance at layer $j > 0$. In this case, the WOTS+ signs the root of layer $j - 1$. In XMSSMT, signature verification is performed by computing the root at each layer and comparing the top-layer root with the public key. If the seed belongs to a layer $j > 0$, the adversary can forge the subtree below layer j for a chosen message, replacing the signature value at layer j while keeping the rest of the signature unchanged. Since the upper layers remain valid, the signature is accepted as legitimate.

Impact and Countermeasures. This attack targets XMSS^{MT} implementations where WOTS+ secret keys are generated deterministically from per-leaf seeds using a simple PRF construction. The attack complexity depends on the number of observable signatures and the probability of zero-valued base- w digits. **Countermeasure:** The attack is prevented by deriving each element of the WOTS+ secret key *independently* from the master seed using a PRF that incorporates the chain index, i.e., $sk_{i,j} = \text{PRF}(SK.\text{seed}, \text{ADRS}_{i,j})$, where the address includes both the OTS index and the chain index. This ensures that recovering one chain value does not reveal the seed and, hence, other chain values.

6.2 Antonov’s Attack

The attack described in [PKC22] exploits the Merkle–Damgård construction of the SHA-256 algorithm. Assume there are six target messages $M_0, \dots, M_5$. For each message, different address values $\text{ADRS}_0, \dots, \text{ADRS}_5$ are used within the hash algorithm. At this stage, intermediate values of the compression function C are obtained for each address.

Then, by applying a collision attack, the adversary finds random values $x_0, \dots, x_5$, and captures pairwise collisions for the next set of intermediate values.

At this point, using the randomly found values $X_{0,1}, X_{2,3}, X_{4,5}$, a 3-branch collision is constructed.

As the final step, the adversary finds a value Z such that the following equation holds

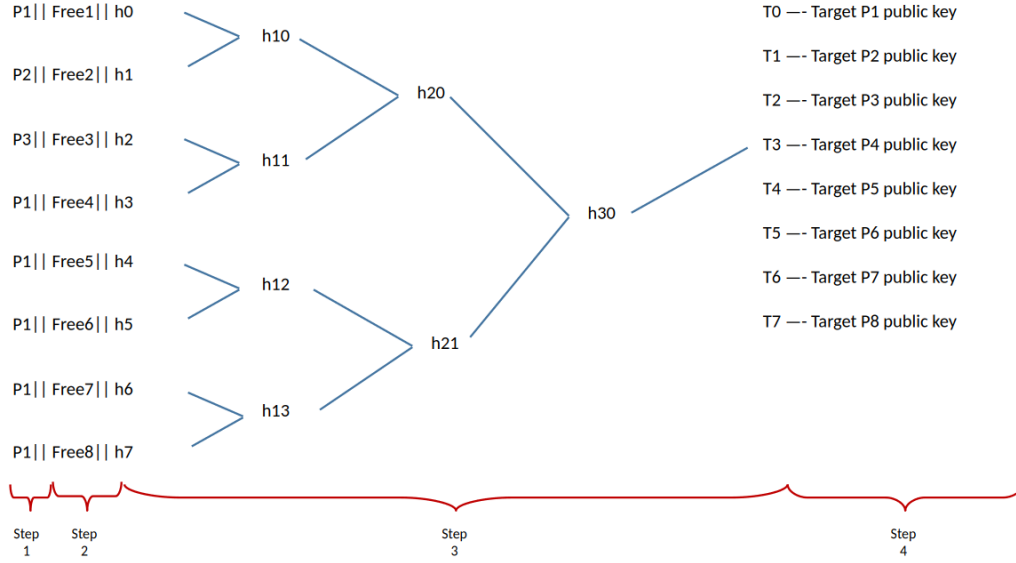


Figure 11: Antonov's Attack

for some $i \in \{0, \dots, 5\}$:

$$\text{SHA-256}(\text{ADRS}_i || x_i || X_{j,k} || Z) = \text{SHA-256}(\text{ADRS}_i || M_i)$$

For example, if the prediction is found for M_3 , then the equation

$$\text{SHA-256}(\text{ADRS}_3 || x_3 || X_{2,3} || Z) = \text{SHA-256}(\text{ADRS}_3 || M_3)$$

is satisfied, thus completing the multi-target prediction attack.

The complexity of finding a prediction in the final stage for t target messages is approximately $2^{256}/t$ compression function calls. The cost of finding an n -way collision is also determined by the number of C function evaluations.

This attack is broken down into four stages, as illustrated in Figure 11

Step 1: In this phase, the adversary collects a number of message-signature pairs signed using the same hash-based signature algorithm. Since the WOTS+ public keys used in the signature can be derived from the signature itself, the adversary also obtains these public keys. The attacker prefers messages with low checksum values at this stage (as lower checksums increase the probability of having F digits in the base- w representation). Although all message-signature pairs share the same PK.seed, each has a distinct AD RS. In Figure 11, $P_1, \dots, P_8$ represent the AD RS and PK.seed values. Finally, the attacker sorts these values.

Step 2: Here, the attacker selects random values $sk_1, \dots, sk_n$ and computes their hashes $2w$ times. By concatenating these values, they obtain the aggregated WOTS+ public keys denoted as $\text{free}_1, \dots, \text{free}_8$.

Step 3: The attacker appends random values h to the free values and uses the method explained at the beginning of this section to find an n -way collision. Unlike Stage 2, the attacker does not know the secret keys corresponding to the WOTS+ public keys in this step.

Step 4: In this stage, the attacker compares the found n -way collision value with the targeted public keys using the value h_{30} . $T_0, \dots, T_7$ represent the target public keys before the checksum is added. If a collision is detected, the attacker has successfully forged a new WOTS+ public key whose hash value matches that of a legitimate public key.

As a result of the above steps, the attacker can generate a forged signature with a fake key. In both SPHINCS+ and XMSS algorithms, public keys generated at each layer during verification are not checked directly—instead, their hash values are computed and passed on to the next layer. Therefore, even if the attacker’s keys differ from the legitimate keys, verification succeeds as long as the hash values match. The main constraint of this attack is that the message to be signed must follow a specific format:

XXXXXXXX XXXXXXXX XXXXXXXF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF

(where X can be any value). This requirement arises because the attacker knows the secret keys corresponding to the public keys used in Stage 2, but not those in Stage 3. In WOTS+ signatures, the public key is used to sign positions marked with F ; therefore, message segments where the secret key is unknown must be fixed to F to successfully carry out the attack.

Impact and Countermeasures. This attack demonstrates that the Merkle–Damgård internal structure of SHA-256 can be exploited in the context of HBS to reduce the cost of multi-target preimage searches. The attack is primarily theoretical and requires a very large number of signature queries. **Countermeasure:** Using hash functions that do not rely on the Merkle–Damgård construction (e.g., SHA-3/SHAKE) eliminates this attack vector. Additionally, choosing larger security parameters increases the attack complexity beyond practical feasibility.

6.3 Fault Injection Attacks

Fault injection attacks consist of scenarios in which one or more components of the algorithm are assumed to malfunction or operate incompletely due to external interference. For hash-based signature schemes, an example of a fault injection attack is described in the [WWO⁺23].

The WOTS+ one-time signature algorithm divides the message to be signed into w -bit chunks and generates the signature value by repeatedly applying the hash function to the secret key as many times as the numerical value of each chunk. At this stage, an attacker can compute the hash of the signature value one more time (when the message is not F) to create a valid signature. To prevent this, a checksum value is appended to the message before signing. This checksum is calculated as $c = (w - 1 - m)$, where m is the number of w -bit message chunks.

With this checksum, if the attacker computes extra hashes from the signature value, the checksum value decreases. Since the hash functions are designed to be resistant to preimage attacks, the attacker cannot compute the new checksum value. The attack described in [WWO⁺23] is based on the checksum calculation process malfunctioning due to an injected fault. If the checksum calculation does not work correctly, the attacker may generate a valid signature for a message of their choice.

Impact and Countermeasures. Fault injection attacks are particularly relevant for embedded implementations of HBS (e.g., in smart cards or HSMs) where physical access may be possible. **Countermeasure:** Implementations should include redundant checksum computation and verification, execute critical operations multiple times and compare results, and employ hardware-level fault detection mechanisms (e.g., error-detecting codes, voltage glitch detectors).

6.4 Multi-Target Attacks

Multi-target attacks aim to find preimages when multiple output values of the used hash function are known. For a hash function with an n -bit output, the complexity of finding a preimage is 2^n . In multi-target attacks, if d different hash outputs are known, the complexity of finding a preimage reduces to $2^n/d$.

The XMSS and SPHINCS+ algorithms mitigate this situation by using keyed hash functions. In each usage of the hash function, different key and mask values are used, effectively differentiating the hash function for each operation and ensuring security against multi-target attacks.

Impact and Countermeasures. Without mitigation, multi-target attacks can significantly reduce the effective security level of HBS schemes, since a tree of height h exposes 2^h hash outputs. **Countermeasure:** Both XMSS and SPHINCS+ use *tweakable hash functions* (via address-dependent keys or masks) to ensure that each hash evaluation is effectively independent, preventing the adversary from amortizing preimage searches across different tree positions. LMS/HSS achieves the same goal through input-formatting-based domain separation.

6.5 Summary of Attacks

Table 6 summarizes the attacks discussed above.

Table 6: Summary of attacks on hash-based signature schemes

Attack	Target	Type	Countermeasure
Intermediate Value	XMSS ^{MT}	Key recovery	Independent PRF per chain
Antonov’s Attack	SPHINCS+/XMSS	Forgery	Larger parameters, non-MD hash
Fault Injection	WOTS+	Impl. attack	Redundant checksum verification
Multi-Target	All HBS	Generic	Tweakable/keyed hash functions

7 Conclusion

Hash-based signature schemes provide a mature and theoretically sound foundation for post-quantum digital signatures. Their security is grounded in the robustness of cryptographic hash functions, making them well-suited for a future where quantum adversaries may render traditional public-key cryptosystems obsolete.

In this paper, we presented a comprehensive overview of hash-based signature constructions, starting from the foundational building blocks—Lamport signatures, Winternitz one-time signatures (WOTS/WOTS+), and Merkle trees—to the standardized schemes XMSS (RFC 8391), LMS (RFC 8554), and the stateless SPHINCS+ (FIPS 205 / SLH-DSA). We analyzed their structural differences, key components, and operational procedures, including the role of checksum mechanisms, address structures, and few-time signature schemes such as FORS. A systematic comparison of these three standardized schemes highlights the trade-offs between statefulness, signature size, and implementation complexity.

We also discussed several cryptanalytic threats specific to HBS schemes—intermediate value guessing, Antonov’s multi-target preimage attack, fault injection attacks, and generic multi-target attacks—analyzing for each the attack model, applicable constructions, complexity, and known countermeasures. These analyses underscore the importance of careful implementation choices (e.g., independent per-chain key derivation, redundant checksum verification) and appropriate parameter selection.

Future work may focus on improving efficiency through optimized tree traversal algorithms, reducing signature sizes via tighter security proofs, exploring the integration of LMS and XMSS into hybrid signature schemes, and designing post-quantum transition strategies that combine the strengths of HBS with other post-quantum approaches such as lattice-based or code-based signatures.

References

- [BDE⁺13] Johannes Buchmann, Erik Dahmen, Sarah Ereth, Andreas Hülsing, and Markus Rückert. On the security of the winternitz one-time signature scheme. *International Journal of Applied Cryptography*, 3(1):84–96, 2013.
- [BHK⁺19] Daniel J Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The sphincs+ signature framework. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 2129–2146, 2019.
- [BHT98] Gilles Brassard, Peter Høyer, and Alain Tapp. *Quantum cryptanalysis of hash and claw-free functions: Invited paper*, page 163–169. Springer Berlin Heidelberg, 1998. URL: <http://dx.doi.org/10.1007/BFb0054319>, doi: [10.1007/bfb0054319](https://doi.org/10.1007/bfb0054319).
- [BXKSN21] Roland Booth, Yanhong Xu, Sabyasachi Karati, and Reihaneh Safavi-Naini. An intermediate secret-guessing attack on hash-based signatures. In *International Workshop on Security*, pages 195–215. Springer, 2021.
- [CAD⁺20] David A Cooper, Daniel C Apon, Quynh H Dang, Michael S Davidson, Morris J Dworkin, Carl A Miller, et al. Recommendation for stateful hash-based signature schemes. *NIST Special Publication*, 800(208):800–208, 2020.
- [HBG⁺18] A Huelsing, D Butin, S Gazdag, J Rijneveld, and A Mohaisen. Rfc 8391: Xmss: extended merkle signature scheme, 2018.
- [Hü17] Andreas Hülsing. WOTS+ – shorter signatures for hash-based signature schemes. Cryptology ePrint Archive, Paper 2017/965, 2017. URL: <https://eprint.iacr.org/2017/965>, doi: [10.1007/978-3-642-38553-7](https://doi.org/10.1007/978-3-642-38553-7).
- [Lam79] Leslie Lamport. Constructing digital signatures from a one way function. Technical Report CSL-98, October 1979. This paper was published by IEEE in the Proceedings of HICSS-43 in January, 2010. URL: <https://www.microsoft.com/en-us/research/publication/constructing-digital-signatures-one-way-function/>.
- [MCF19] David McGrew, Michael Curcio, and Scott Fluhrer. Rfc 8554: Leighton-micali hash-based signatures. [urlhttps://www.rfc-editor.org/rfc/rfc8554](https://www.rfc-editor.org/rfc/rfc8554), 2019.
- [NIS24] Gaithersburg MD NIST. Stateless hash-based digital signature standard. *NIST FIPS*, 01 2024. doi: [10.6028/nist.fips.205](https://doi.org/10.6028/nist.fips.205).
- [PKC22] Ray Perlner, John Kelsey, and David Cooper. Breaking category five sphincs+ with sha-256. In *International Conference on Post-Quantum Cryptography*, pages 501–522. Springer, 2022.

- [WWO⁺23] Alexander Wagner, Vera Wesselkamp, Felix Oberhansl, Marc Schink, and Emanuele Strieder. Faulting winternitz one-time signatures to forge lms, xmss, or signatures. In *International Conference on Post-Quantum Cryptography*, pages 658–687. Springer, 2023.